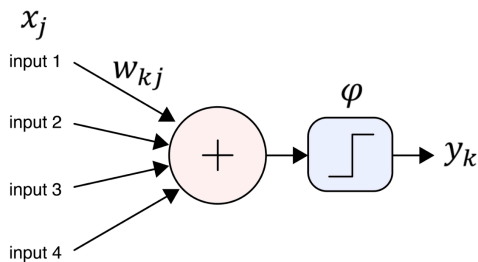
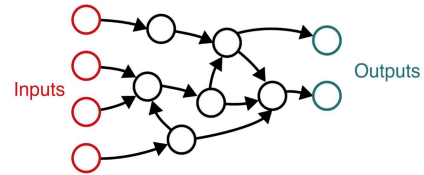


Clase XXI: Introducción a las Redes Neuronales

Red → grafo dirigido sin ciclos, con nodos y ejes.

También se llaman **feedforward neural networks**.

¿Por qué neuronal? → Porque modela una neurona.



Perceptrón → neurona artificial que puede con la clasificación lineal.

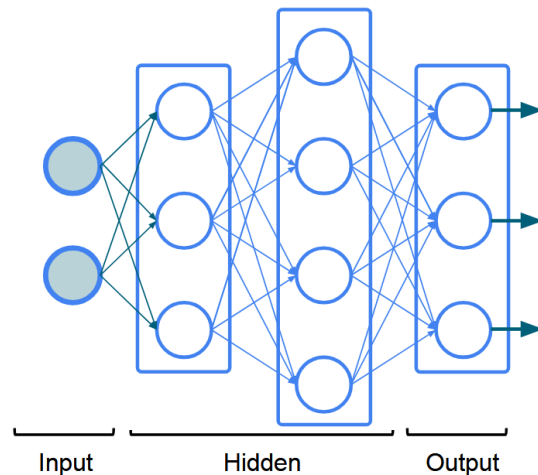
- 1) **Input:** número (escalar)
- 2) Cada input se **multiplica por un peso**.
- 3) Se **suman** los resultados.
- 4) Se **aplica** una función de **activación**.
- 5) (Opcional) se puede **incluir un bias**.
- 6) Fórmula:

$$y_k = \varphi\left(\sum_j w_{kj} x_j\right)$$

Las redes neuronales pueden combinar capas con funciones no lineales como una composición de varias funciones

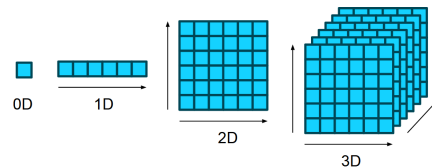
Perceptrón Multicapa (MLP):

- 1) Agrupa neuronas en **layers**.
 - a) Input layer
 - b) Hidden layers
 - c) Output layer
- 2) **Todas las layers están conectadas.** Se forma un grafo completo.
- 3) La capacidad de una red depende del número de nodos y capas.
 - a) Red shallow: pocas layers y más anchas
 - b) Red deep: muchas layers y más profundas.



Tensores:

- 1) Arreglos multidimensionales de escalares.
 - a) Escalar → 0 dimensiones
 - b) Vector → 1 dimensión
 - c) Matriz → 2 dimensiones
 - d) Tensor → n dimensiones
- 2) Representan datos de las redes neuronales: inputs, outputs, weights y biases.
- 3) Permiten trabajar con datos complejos (imágenes, videos) y procesamiento paralelo en GPU.

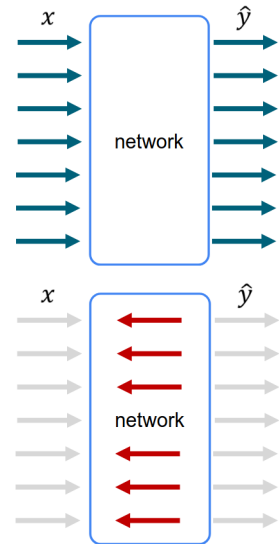


Clase XXII: Forward y Backward Propagation

Entrenamiento de redes neuronales → modificamos los pesos y los biases para que aprendan los datos.

El entrenamiento consiste de:

- 1) **Forward propagation.**
 - a) **Flujo** visto → input x / hidden layers / output.
 - b) Se **calcula el output** para una entrada con los weights y los biases.
 - c) Output → la **predicción** $\hat{y}$.
- 2) **Backward propagation.**
 - a) **Flujo** → output / hidden layers / input x .
 - b) Se calculan los **gradientes** de la función loss respecto a los weights y biases.
 - c) Después se **actualizan los parámetros** para minimizar el error.
 - d) Flexibilidad de entrenamiento entre capas
 - i) **Capas entrenables** → weights y biases se pueden ajustar en backward propagation
 - ii) **Capas no entrenables** → se fijan durante el entrenamiento para mantener características ya aprendidas.



Cada unidad k tiene $z_k = \sum_j w_{kj} + b_k$ y $a_k = \varphi(z_k)$.

→ w_{kj} es el **weight** del input j a la neurona k .

→ b_k es el **bias** de la neurona k .

→ φ es la **función de activación**.

→ a_k es la **neurona activada**.

Ejemplo: dos inputs y tres unidades.

Entrada total de las neuronas → **input neto**:

$$z_1 = w_{11}x_1 + w_{21}x_2 + b_1$$

$$z_2 = w_{12}x_1 + w_{22}x_2 + b_2$$

$$z_3 = w_{13}x_1 + w_{23}x_2 + b_3$$

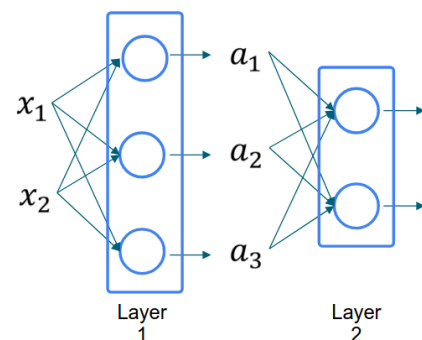
Se aplican luego las **activaciones**:

$$a_1 = \varphi(z_1)$$

$$a_2 = \varphi(z_2)$$

$$a_3 = \varphi(z_3)$$

Cada layer toma las activaciones de la anterior como entrada.



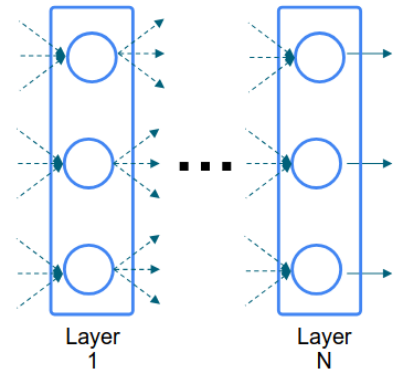
Vectorización y cálculos entre capas:

- 1) Las **capas fully connected** conectan todas las neuronas de una capa con todas las neuronas de la capa siguiente.
- 2) Se usa la notación matricial.

$$a) \quad z = W^T x + b$$

- i) W es una matriz de pesos
- ii) x es un vector de inputs
- iii) b es un vector de biases

$$b) \quad a = \varphi(z)$$



Función de pérdida → **loss**

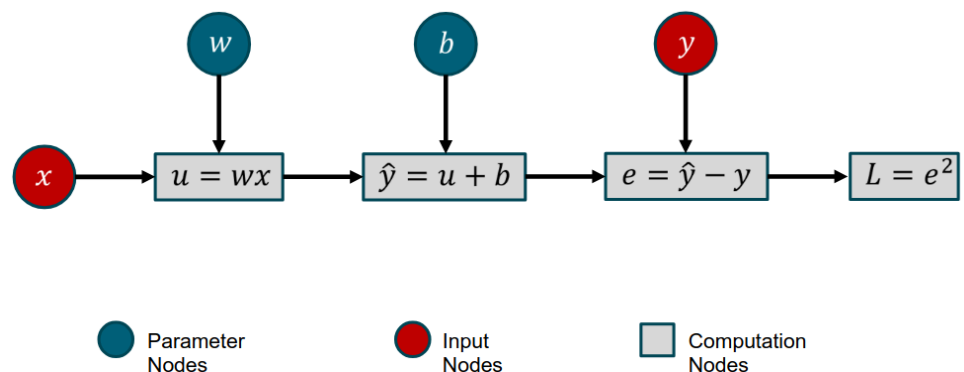
- 1) Cuantifica el error entre la predicción y la etiqueta real.
- 2) Guía para entrenar.
- 3) **MSE** para regresión.
- 4) **Cross-Entropy** para clasificación.

Backward Propagation:

- 1) La loss mide la diferencia → **actualiza** pesos y bias → **minimizar pérdida**.
- 2) Propaga el error **para atrás**.

Grafos Computacionales:

Representación de todas las **operaciones como nodos**.



Representación de $((wx + b) - y)^2$.

- Se pueden reutilizar cálculos ya hechos.
- Se puede usar también la regla de la cadena.

Batch Processing:

- 1) Una red neuronal puede procesar muchas muestras (conjunto = **batch**) en simultáneo
- 2) Una muestra Z se representa como $Z = W^T X + b$.
 - a) X puede contener varias observaciones.
- 3) Conseguimos cálculos eficientes y paralelizables.

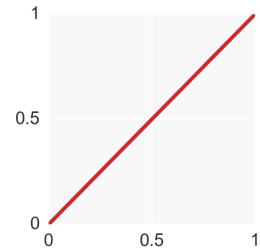
Clase XXIII: Funciones de Activación

Funciones de activación:

- 1) Introducen mapeos no lineales en la red neuronal.
- 2) No hay activación → una red se colapsa a una transformación lineal.

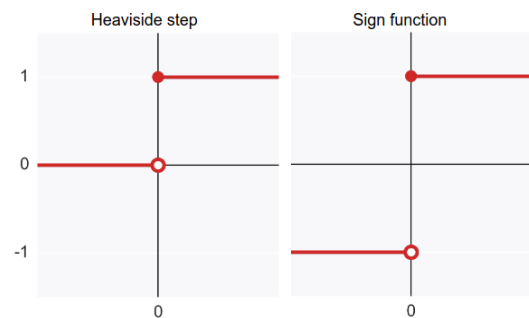
Funciones de activación lineales:

- 1) Puede ser la función de identidad → $f(x) = x$
- 2) Input = output.
- 3) (+) Son simples y eficientes.
- 4) (-) Pueden colapsar la red neuronal → se usan solo en la capa de salida.
- 5) Se puede cambiar la ordenada al origen y la pendiente, que el efecto va a ser el mismo.



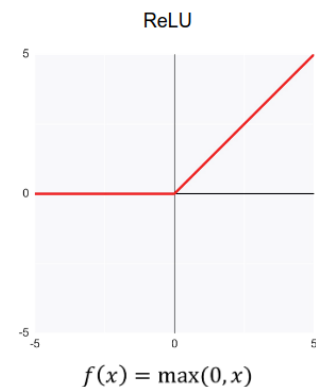
Funciones escalonadas (Step):

- 1) La función no-lineal más simple.
- 2) Valores fijos → según un threshold.
 - a) $\{0, 1\} \rightarrow$ **Heaviside Step**
 - b) $\{-1, 1\} \rightarrow$ **Sign Function**
- 3) (+) Simples y sirven para salidas binarias.
- 4) (-) No diferenciables → no funciona gradiente descendente.



Funciones lineales por partes (Piecewise):

- 1) Tiene partes lineales.
- 2) (+) Eficientes y ayudan al vanishing gradient.
- 3) (-) Mete más hiperparámetros.
- 4) Input negativo → output 0.
- 5) Input positivo → función identidad.
- 6) La más usada → **ReLU**.
 - a) Problema → Dying ReLU. Aparecen neuronas muertas con gradiente 0.
 - b) Variantes:
 - i) Leaky ReLU
 - ii) Shifted ReLU
 - iii) Maxout



Funciones suaves:

Permiten gradientes continuos → optimización más estable.

- 1) **Softplus** → ReLU suave
 - 2) **ELU** → Shifted ReLU suave → sin neuronas muertas
 - 3) **Sigmoide** → salida entre 0 y 1 → para clasificación con probabilidades → sufre vanishing gradient.
 - 4) **Tangente hiperbólica** → centrada en 0 y salida acotada entre -1 y 1.
 - 5) **Swish** → ReLU + Sigmoide
- (+) Buenas para convergencia (-) Costosas y algunas sufren vanishing gradient.

Función Softmax:

- 1) Es una operación que se aplica solamente en todas las neuronas de output de un clasificador.
- 2) No es solamente una función de activación.
- 3) **Normaliza** un vector de entrada x con K números a una distribución de probabilidad $\rightarrow$ genera K probabilidades.
- 4) Existe la modificación **Log-Softmax**.



Clase XXIV: Optimizadores

Deep learning $\rightarrow$ minimizar una función loss.

El **gradiente** indica la **dirección** en la que cambia la función $\rightarrow$ para minimizar, se avanzan pasos en el sentido opuesto al gradiente.

Error Landscape:

- 1) Es la geometría de la función loss.
- 2) Posibles representaciones:
 - a) Máximos y mínimos
 - b) Mesetas
 - c) Sillas de montar (forma Pringles)
- 3) Problema frecuente $\rightarrow$ **vanishing gradient**.

Learning Rate (LR):

- 1) Es un hiperparámetro ϵ .
- 2) Controla el tamaño del paso para actualizar los pesos.
 - a) ϵ grande $\rightarrow$ el modelo diverge.
 - b) ϵ chico $\rightarrow$ aprende lento el modelo.
- 3) La trayectoria en el espacio de parámetros depende del LR.

Schedules:

- 1) Es el ajuste del LR en el tiempo. LR no es constante siempre.
- 2) Tipos de schedules:
 - a) **Decaimiento exponencial.**
 $\rightarrow \epsilon_k = \epsilon_0 \alpha^k$
 - b) **Decaimiento exponencial retrasado.**
 $\rightarrow$ LR **constante** y luego **decrece**
 - c) **Decaimiento basado en el error.**
Disminuye LR mientras baja el error

Tipos de Descenso Gradiente:

Tipo	Descripción	Ventajas	Desventajas
Batch (BGD)	Usa todo el dataset	Estable y preciso	Costoso y lento
Estocástico (SGD)	Usa un ejemplo por paso	Rápido y simple	Muy ruidoso e inestable
Mini-Batch	Usa pequeños grupos	Costo y precisión parejos	Depende del tamaño del batch

Momentum:

- 1) Agrega inercia al update del LR → suaviza la trayectoria y ayuda a escapar de las mesetas.

$$\rightarrow v_t = \alpha v_{t-1} - \epsilon \Delta f(x_t)$$

$$\rightarrow x_{t+1} = x_t + v_t$$
- 2) El parámetro α controla cuánto influye el gradiente anterior.

Learning Rates Adaptativos:

Optimizador	Descripción	Ventajas	Desventajas
AdaGrad	Escala el LR inversamente a la raíz de la suma de gradientes acumulados	Mejora en regiones curvas	El gradiente se hace muy chico
Adadelta / RMSprop	Usa la suma decay	Converge más rápido	Usa el nuevo hiperparámetro → decay
ADAM	Combina promedios de 1er y 2do momento (gradiente y su cuadrado)	Muy eficiente y común	Usa dos hiperparámetros más (β_1 y β_2)

- No hay un optimizador mejor que todo el resto.
- La elección depende del modelo, los datos y los hiperparámetros.
- Se elige el optimizador evaluando distintas combinaciones.

Clase XXV: Parámetros e Hiperparámetros

Underfitting → el modelo no logra bajar el error de entrenamiento porque el modelo es muy simple. Es un modelo de baja capacidad.

Overfitting → el error en el test es mucho mayor que en el entrenamiento. Es un modelo de alta capacidad.

→ Queremos buscar el punto justo entre los dos.

Bias → diferencia entre predicciones y valores reales.

Un **bias alto** implica que tenemos un **modelo muy simple** y, por ende, **underfitting**.

Varianza → sensibilidad del modelo a los datos de entrenamiento.

Una **varianza alta** implica que tenemos un **modelo inestable** y que tiene **overfitting**.

Reglas generales:

- 1) + Capacidad → + Varianza, – Bias
 - 2) – Capacidad → – Varianza, + Bias
- La capacidad del modelo determina cuánto puede aprender y qué tan bien generaliza con datos nuevos.

Parámetro → **aprendido durante** el entrenamiento.

Hiperparámetro → se **define antes** del entrenamiento.

Elección de Hiperparámetros:

- 1) **Learning Rate (LR):**
 - a) **Bajo** → lento y estable.
 - b) **Alto** → rápido e inestable.
 - c) Usar **scheduling** → ajustar LR durante el entrenamiento.
- 2) **Capas ocultas** → más capas implica más complejidad y riesgo de overfitting.
- 3) **Unidades ocultas:**
 - a) **Pocas** → no aprende bien.
 - b) **Muchas** → overfitting y entrenamiento lento.
 - c) Conviene usar la media geométrica entre unidades de entrada y de salida.
- 4) **Funciones de activación:**
 - a) ReLU → común y sin vanishing gradient.
 - b) Sigmoide → para salidas binarias.
- 5) **Momentum o Mini-batch:**
 - a) **Momentum** → acelera y evita mínimos locales.
 - b) **Mini-batch** → afecta la estabilidad y el costo computacional.

Búsqueda de Hiperparámetros:

- 1) **Grid Search:**
 - a) Prueba todas las combinaciones posibles. Es simple pero costoso computacionalmente.

- 2) **Random Search:**
 - a) Toma combinaciones al azar.
 - b) Más eficiente que GS.
 - c) No garantiza el óptimo.
- 3) **Optimización Bayesiana:**
 - a) Evaluaciones previas para guiar la búsqueda.
 - b) Eficiente y adaptativa.
 - c) Compleja de implementar.

Training set → ajuste de parámetros.

Validation set → elección de hiperparámetros.

Test set → evaluación de generalización.

Conviene mantener distribuciones similares entre estos sets.

Normalizaciones:

- 1) **Batch Normalization:**
 - a) Normaliza por mini-batch.
 - b) Evita valores extremos y mejora la convergencia.
- 2) **Layer Normalization:**
 - a) Normaliza entre atributos por muestra.
 - b) Útil en RNNs y transformers.

Problemas con Gradientes:

- 1) **Vanishing Gradient:**
 - a) Gradientes muy pequeños.
 - b) Aprendizaje lento.
- 2) **Exploding Gradient:**
 - a) Gradientes grandes.
 - b) Entrenamiento inestable.

Se solucionan estos problemas con buenas inicializaciones de pesos.

Monitoreo de pérdidas → observando training loss y validation loss.

Si **divergen las dos** pérdidas → **overfitting**.

Si son **altas las dos** pérdidas → **underfitting**.

Monitoreo de gradientes → detección de problemas de estabilidad o LR.

Regularización:

- 1) **L1:**
 - a) Penaliza **valores absolutos**.
 - b) Genera modelos simples y esparsos.
- 2) **L2:**
 - a) Penaliza **cuadrados**.
 - b) Genera pesos más suaves y distribuidos.
- 3) **Dropout:**
 - a) **Desactiva neuronas** aleatoriamente.
 - b) Mejora la generalización.
- 4) Otros: **Data augmentation, noise injection, early stopping.**

Clase XXVI: Redes Neuronales Convolucionales

Las **Redes Neuronales Convolucionales (CNN)** se usan en los siguientes campos:

- 1) Autenticación biométrica.
- 2) Imágenes médicas.
- 3) Análisis de documentos.
- 4) Conducción autónoma.
- 5) Generación multimedia.

Convolución:

- 1) Es una **operación matemática**.
 - a) Toma dos funciones o matrices.
 - b) Genera una tercera función.
 - c) Mide cómo se diferencia una función (f) cuando interactúa con la otra (g).
- 2) Notación con un **asterisco ***.
 - a) $f(x) \rightarrow$ **input**.
 - b) $g(x) \rightarrow$ **kernel**, si el input es una imagen. Es una matriz chica.
 - i) En un caso discreto, el kernel recorre el input multiplicando y sumando los valores.
 - c) $(f * g)(x) \rightarrow$ **convolución** $\rightarrow$ transforma el input en un **feature map** que resalta las características del input.
- 3) Puede ser **continuo** o **discreto**.

$$(f * g)(t) = \int f(a) \cdot g(t - a) da$$

Continuo

$$(f * g)(t) = \sum f(a) \cdot g(t - a)$$

Discreto

Convoluciones Multidimensionales:

- 1) ML $\rightarrow$ inputs y kernels son tensores (arreglos multidimensionales).
- 2) Entonces, se pueden hacer convoluciones sobre más de una dimensión a la vez.
- 3) Ventajas:
 - a) Menos parámetros.
 - b) Pesos compartidos $\rightarrow$ el mismo kernel se aplica en toda la imagen.
 - c) **Cross-Correlation** $\rightarrow$ si el objeto se desplaza, la activación también lo hará.

$$F_{ij} = (I * K)_{ij} = \sum_m \sum_n I_{i+m, j+n} \cdot K_{m,n}$$

Capa Fully Connected vs. Capa Convolutiva:

Capa FC $\rightarrow$ conecta todas las neuronas $\rightarrow$ muchísimos parámetros.

Capa convolutiva $\rightarrow$ usa solo los pesos del kernel $\rightarrow$ pocos parámetros.

Filtro → compuesto por **varios kernels** → uno por canal de entrada.

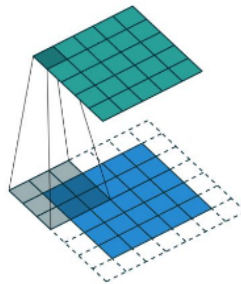
Cada filtro produce un feature map.

Por cada filtro hay una representación distinta del input.

$$F_j^n = b_j + \sum_{k=0}^{c^{in}_{n-1}} F_k^{n-1} * w_{jk}^n$$

$\downarrow$ Output feature maps (j) $\downarrow$ Bias term $\downarrow$ Input feature maps (k) $\downarrow$ Kernels

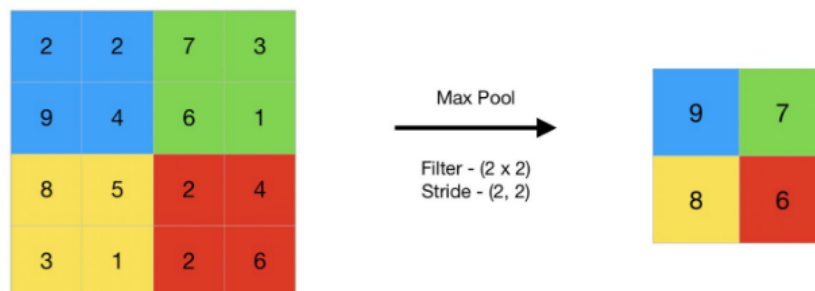
Padding → agrega bordes de píxeles extras para mantener el tamaño espacial de la salida y no perder información de los bordes.



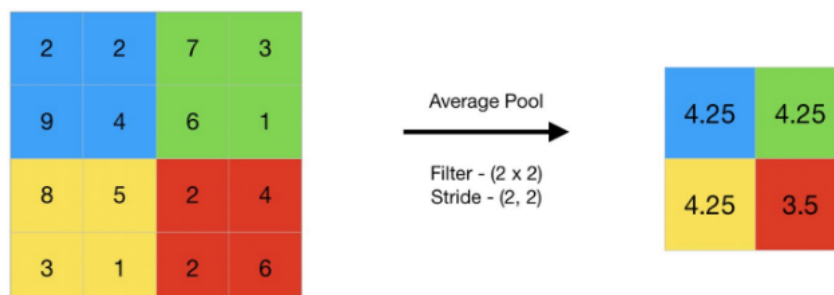
Stride → controla el desplazamiento del kernel. Afecta el tamaño del output y la cantidad de downsampling.

Pooling:

- 1) Reduce la dimensión espacial → downsampling.
- 2) Reemplaza un conjunto de valores por un resumen estadístico.
 - a) Max pooling → toma el máximo valor.

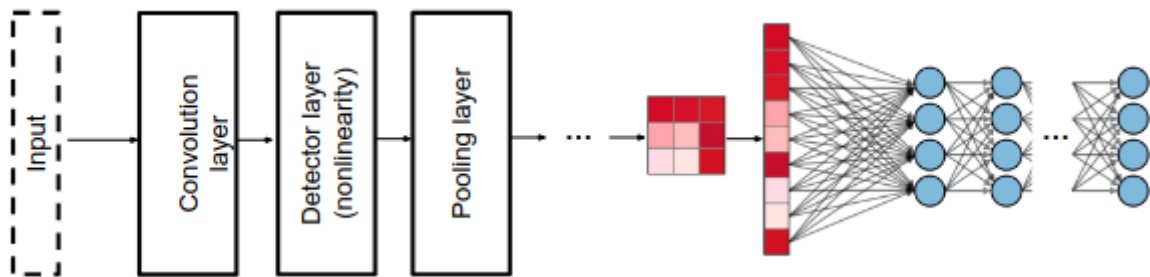


- b) Average pooling → toma el promedio.



- 3) Ventajas:
 - a) Más robusto a traslaciones.
 - b) Reduce el cálculo de las capas siguientes.

Arquitectura de una CNN:



- Las **capas convolucionales iniciales** aprenden bordes o direcciones.
- Las **capas convolucionales intermedias** aprenden texturas o formas simples.
- Las **capas convolucionales finales** aprenden partes de objetos o patrones complejos.

Campo Receptivo:

- 1) Es la zona del input que ve una neurona.
- 2) En **capas bajas** → los campos pequeños detectan detalles simples.
- 3) En **capas altas** → los campos grandes detectan patrones globales.

Clase XXVII: Clasificación de Imágenes, Arquitecturas Famosas y Mapas de Activación

- Las CNN se para entrenar muchos conjuntos de imágenes (como **ImageNet**).

Las **tareas de reconocimiento** se clasifican en:

- 1) **Clasificación** → una etiqueta por imagen.
 - Dice qué hay en la imagen pero no dónde.
- 2) **Clasificación con localización** → ¿dónde está el objeto?
 - Dice qué hay en la imagen y dónde. Uno solo.
- 3) **Detección** → reconocimiento de objetos y posiciones.
 - Dice qué objetos hay en la imagen y dónde. Pueden ser varios.



Clasificación de imágenes

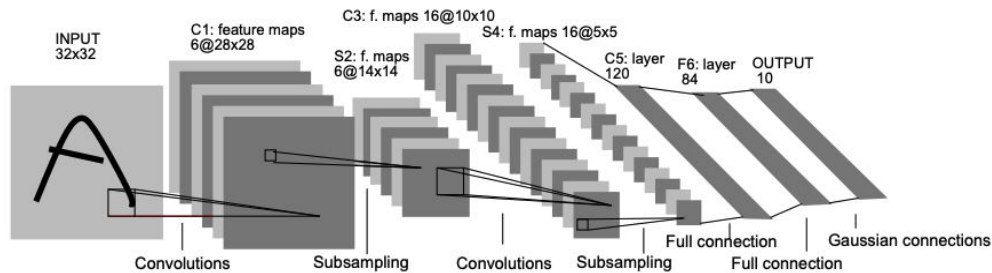
Clasificación con localización

Detección

Arquitecturas famosas:

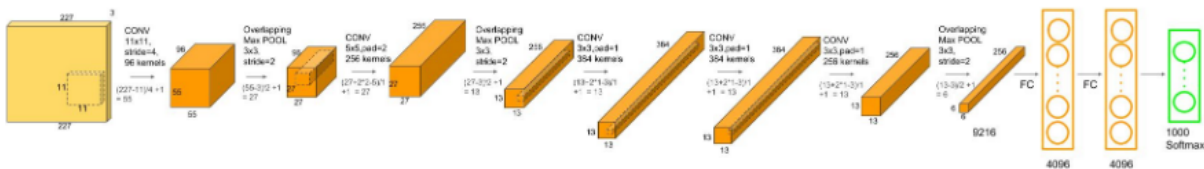
1) LeNet (1998):

- Es una de las primeras CNN.
- Se usa para **reconocimiento de dígitos** → uso de dataset MNIST.
- Muestra cómo pasar de una imagen grande a varias representaciones más chicas → **uso del kernel para la extracción de features**.
 - La red mira pequeñas regiones de la imagen,
 - aprende patrones locales (bordes, líneas, texturas)
 - y codifica la información en mapas más chicos.



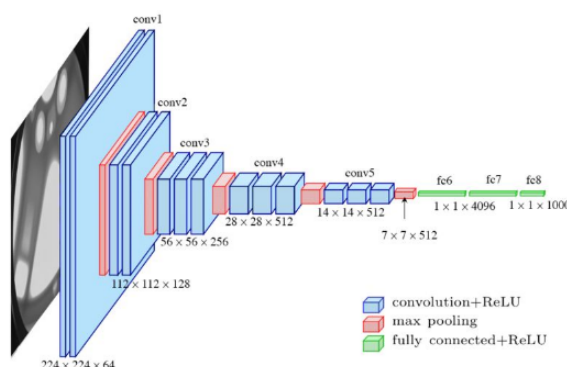
2) AlexNet (2012):

- Uso efectivo de las GPUs.
- Función de activación → ReLU**. Evita saturación y acelera el training.
- Usa **dropout** → técnica para reducir el overfitting.
- Usa técnicas de aumentación de datos → mayor generalización del modelo.
- Preprocesamiento** para mejorar estabilidad y rendimiento → **normalización en las entradas**.
- Hiperparámetros:
 - Mini-batch**,
 - Momentum**,
 - Decay**,
 - Inicialización de pesos no nula, y
 - LR**.



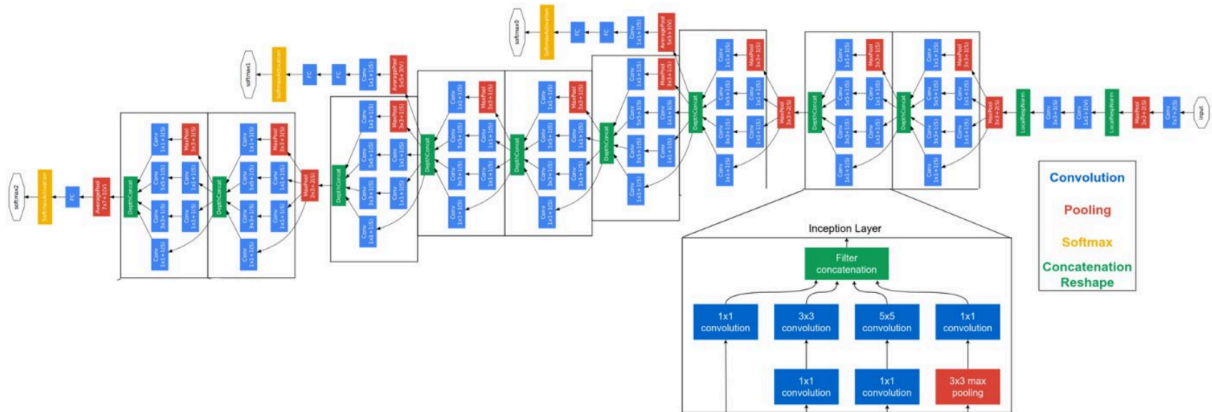
3) VGGNet (2014):

- Estudia **cómo la profundidad** (más capas convolucionales) **afecta a la performance**.
- Uso de **kernels chicos**. Reduce la cantidad de parámetros.
- Training:
 - Uso de **penalización L2** → **weight decay**. Para que los pesos no crezcan mucho.
 - Sin local response normalization**. Diferencia con AlexNet.
 - Preprocesamiento** → restar la media RGB del training set.



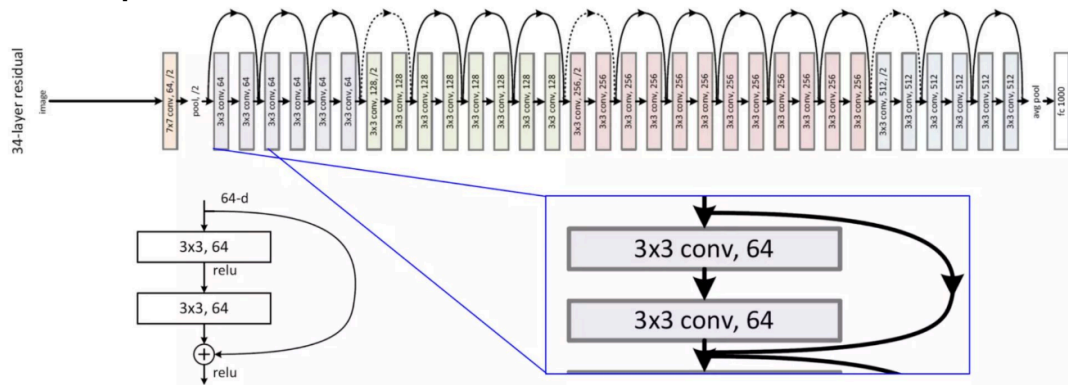
4) GoogLeNet (2014) e Inception:

- Más **rápida** y más **eficiente** que AlexNet.
- Inception Layer** → varias convoluciones en paralelo para concatenar sus outputs.
 - Primero se hacen **convoluciones de reducción de 1x1** antes de las otras → ahorra memoria.
- Apilado de **capas de Inception** con max pooling → stride = 2.
- Average Pooling en lugar de FC** → anti overfitting.
- Uso de loss intermedias para evitar vanishing gradient.



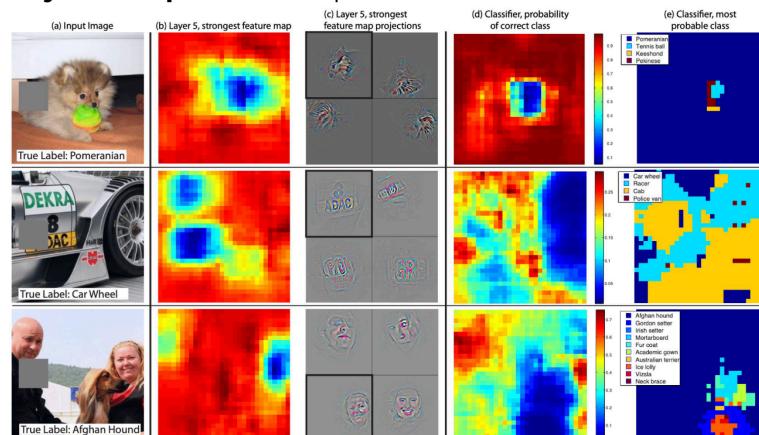
5) ResNet (2015):

- Las redes profundas mejoran la representación → pero pueden sufrir vanishing gradient.
- Usa conexiones residuales (**skip connections**) → para que los gradientes fluyan directamente.
- Bloques residuales apilados** → redes más profundas sin bajar rendimiento.
- Go deep or go home → **red más profunda con ResNet, mejor representación.**



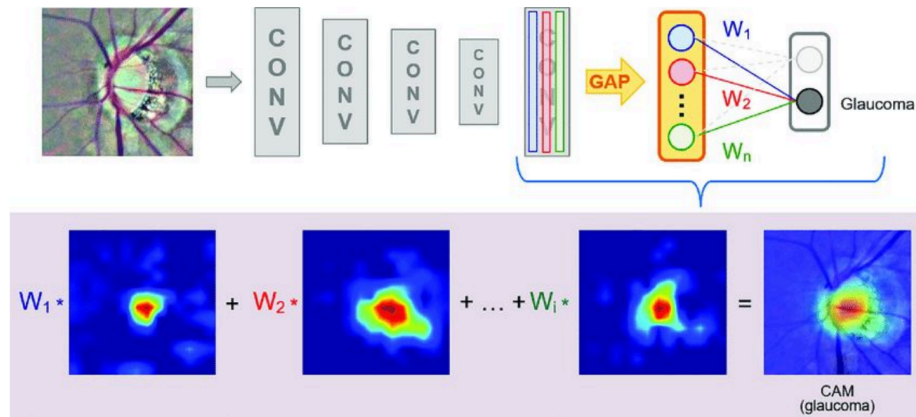
6) ZFNet (2013):

- Extensión** de AlexNet con **mejor visualización.**
- Mejor ajuste de parámetros** para entender las activaciones internas.



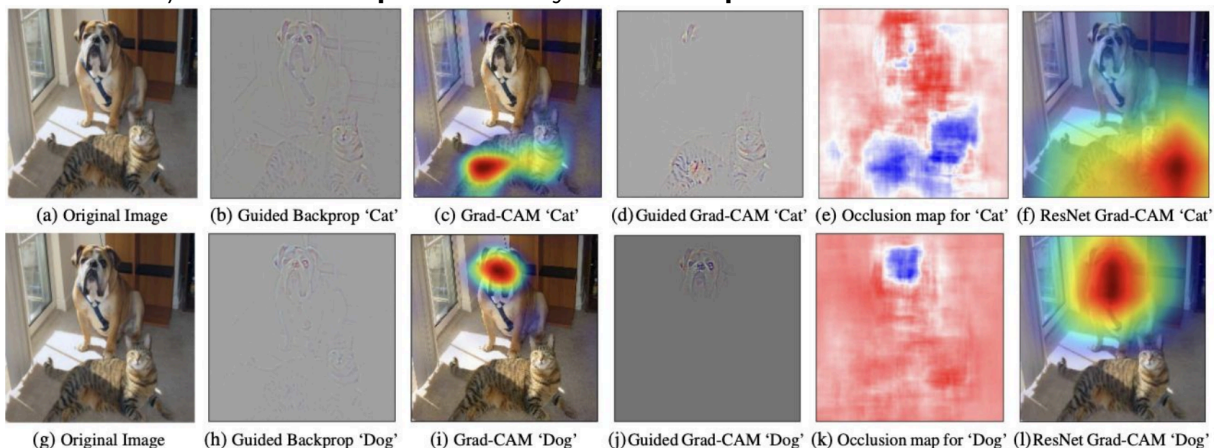
7) CAM – Class Activation Map:

- Muestra **qué partes de la imagen son más importantes para predecir** una clase.
- En training → **la red aprende los pesos por clase para cada feature map**.
- Suma ponderada de esas activaciones** → genera un mapa de activación que resalta regiones relevantes.
- Global Average Pooling (**GAP**) **antes** del softmax.



8) Grad-CAM:

- Variante generalizada del CAM.
- Usa **gradientes del score de clase respecto al último feature map**.
- Resalta las **zonas con mayor influencia en la decisión final**.
- Permite **interpretar CNNs** y **visualizar qué ven estas** al clasificar.



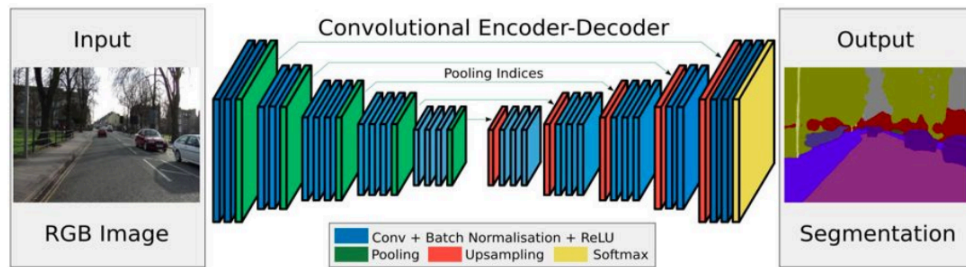
Clase XXVIII: Convoluciones Traspuestas, Segmentación y Decoders

Si pasamos una imagen como input, probablemente querramos que haya otro output que no sea un número o una etiqueta, como otra imagen.

Estos modelos más sofisticados necesitan:

- Encoder:** imagen → representación compacta (vector de características)

- 2) **Decoder:** representación compacta → imagen. Hace la reconstrucción o genera una salida visual.



Podemos elegir cuál es la resolución final de una imagen cuando pasa por una red neuronal. Input → imagen, output → “misma” imagen.

Bajar resolución → pooling.

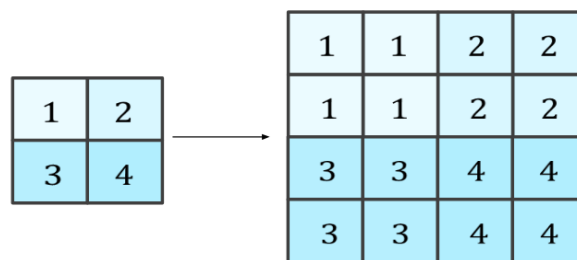
Mantener resolución → padding = 1, stride = 1.

Aumentar resolución (upsampling) → interpolación o convolución transpuesta.

La **interpolación** es un método para aumentar la resolución de una imagen generando nuevos píxeles en base a los píxeles que se le pasan por parámetro. Es una **fórmula fija**. Hay dos tipos de interpolación:

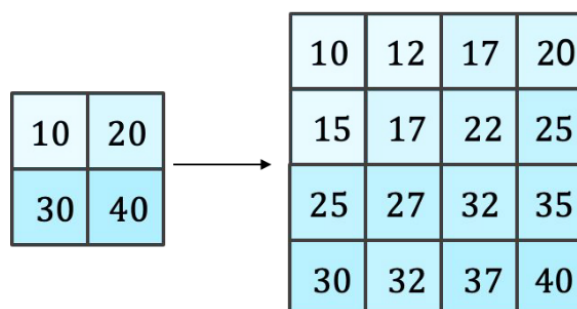
- 1) **Vecino más Cercano:**

- a) Cada nuevo píxel toma el valor del píxel existente más cercano.
b) Rápida y simple.



- 2) **Bilineal:**

- a) Cada nuevo píxel es un promedio ponderado de los 4 píxeles vecinos.
b) Resultados más suaves.

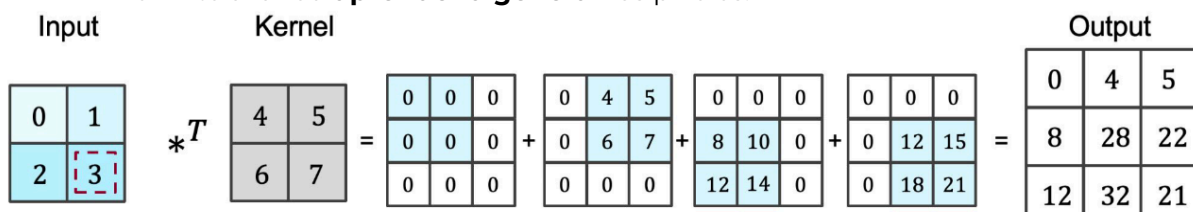


La **convolución transpuesta** hace el proceso inverso a la convolución → **aprende el upsampling** (con el interpolado no se podía).

→ **Parámetros:** stride, padding.

→ **Expande** la imagen de salida. Una convolución normal la disminúa.

→ Permite a la red **aprender a generar** los píxeles.



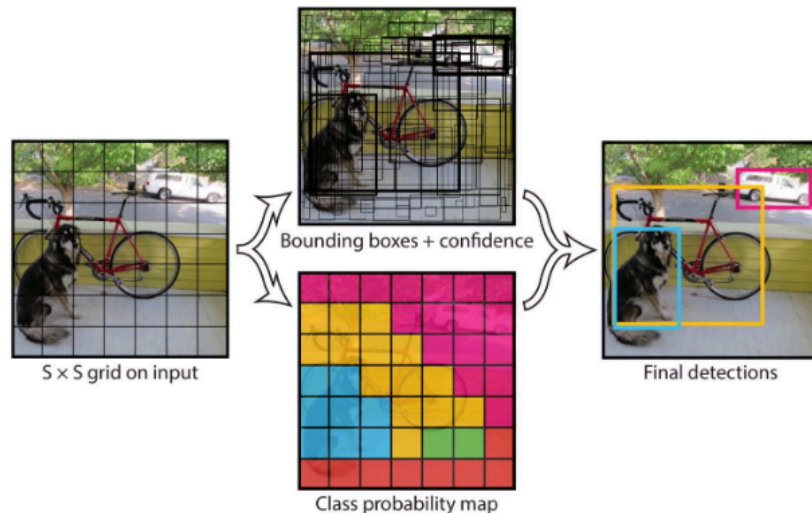
En la **segmentación de imágenes** buscamos predecir una etiqueta por píxel. Vamos a ver 5 tipos de segmentación.

1) **U-Net:**

- a) Arquitectura *encoder-decoder* con conexiones entre capas simétricas.
- b) Última capa: convolución $1 \times 1 \rightarrow$ transforma *feature maps* en clases.
- c) Usa **softmax** final y **cross-entropy loss**.

2) **YOLO (You Only Look Once):**

- a) Divide la imagen en una grilla.
- b) Predice para cada celda los objetos y los atributos.
- c) Usa la supresión de no-máximos para que no haya solapamientos.
- d) Optimiza con una función de pérdida que combina tamaño de bounding box, confianza y probabilidad de clase.

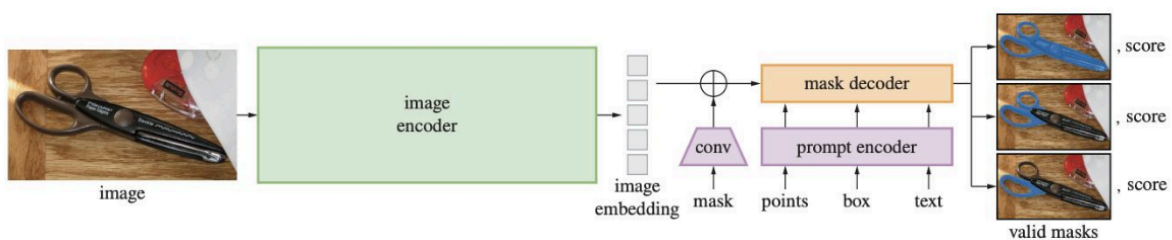


3) **R-CNN (Region-based CNN):**

- a) Genera regiones candidatas y extrae características.
- b) Usa una red para clasificar el objeto en cada región.

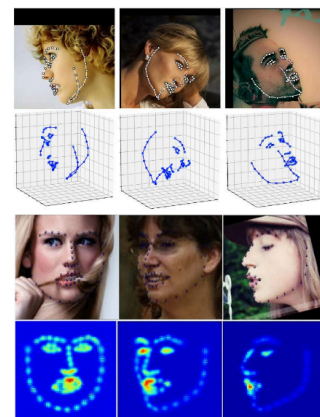
4) **SAM (Segment Anything Model):**

- a) Modelo de segmentación general entrenado con un dataset masivo.
- b) Permite segmentar cualquier objeto de una imagen con un prompt (texto o punto).



5) **FAN (Face Alignment Network):**

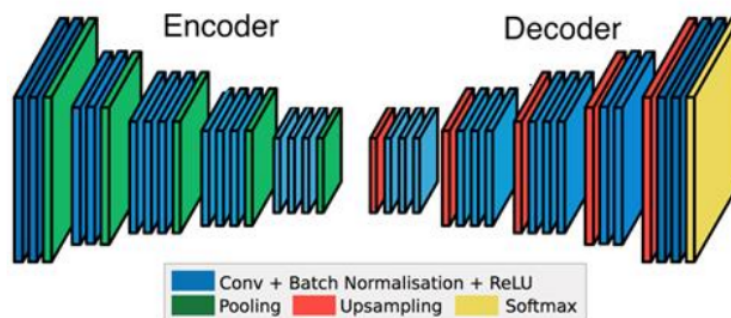
- a) Detecta **landmarks faciales** (puntos de interés).
- b) Usa convoluciones + capas FC.
- c) Predice **heatmaps** que indican la probabilidad de cada punto.
- d) Basado en arquitectura **Hourglass**.



Clase XIX: Autoencoders, VAEs y GANs

Autoencoder:

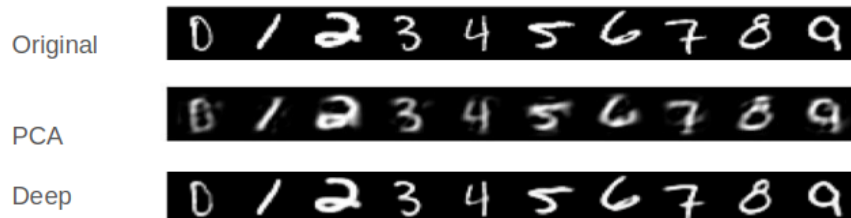
- 1) Es una red neuronal.
- 2) **Objetivo** → aprender una representación comprimida de los datos.
- 3) Toma una entrada x → busca **reconstruirla** como $\hat{x}$.
- 4) Arquitectura:
 - a) **Encoder** → reduce la dimensionalidad y consigue un código latente compacto.
 - b) **Decoder** → reconstruye la entrada original a partir del código.



El **cuello de botella** fuerza a la red a aprender patrones esenciales en vez de copiar los datos. Es la capa entre el encoder y el decoder, donde hay menos neuronas.

Deep autoencoders → permiten aprender manifolds no lineales más expresivos que métodos lineales como PCA. Un **manifold** es un espacio de menor dimensión que está incrustado dentro de un espacio de mayor dimensión.

Limitación → no son generativos, porque no modelan la distribución de los datos ni incorporan incertidumbre.



Variational Autoencoders (VAEs):

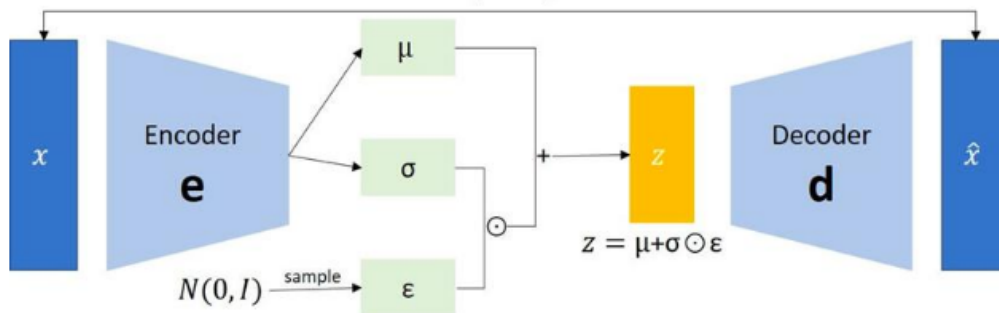
→ Funciona como una **extensión de los autoencoders clásicos** mediante una formulación probabilística.

- 1) El encoder produce las distribuciones (media y varianza) sobre el espacio latente.
- 2) El modelo hace que las distribuciones se parezcan a una distribución normal → $N(0, 1)$.
- 3) Si se parece a una normal, se pueden muestrear puntos del espacio latente y también generar nuevos datos.

→ El **decoder** funciona como **modelo generativo**: se samplea $z \sim N(0, 1)$ para generar x .

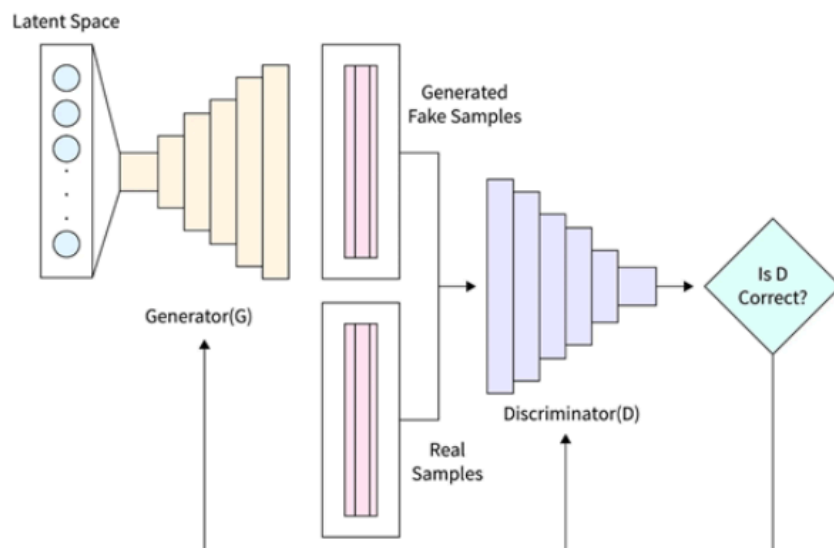
→ **Parametrización** para gradientes estables.

→ Permiten **generar imágenes** y **explorar el espacio latente**.



Generative Adversarial Networks (GANs):

- 1) Tiene **dos redes entrenadas** en conjunto:
 - a) **Generador** → producción de muestras falsas $g(z)$.
 - b) **Discriminador** → estima probabilidad de que una imagen sea real.
- 2) **Entrenamiento** → juego minimax donde:
 - a) El generador intenta engañar al discriminador.
 - b) El discriminador intenta distinguir muestras reales de las generadas.
- 3) **Procedimiento** → alternar entre:
 - a) Entrenar el **discriminador** → **ascenso del gradiente**.
 - b) Entrenar el **generador** → **descenso del gradiente**.

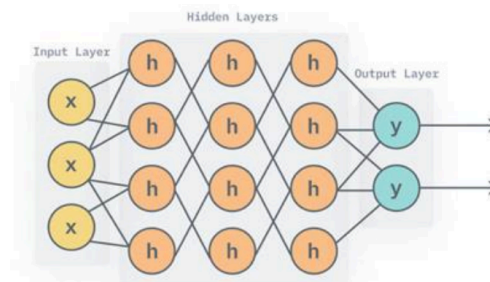


Clase XXX: RNN y LSTM

Redes Feedforward → procesan información solo hacia **adelante** (input a output).

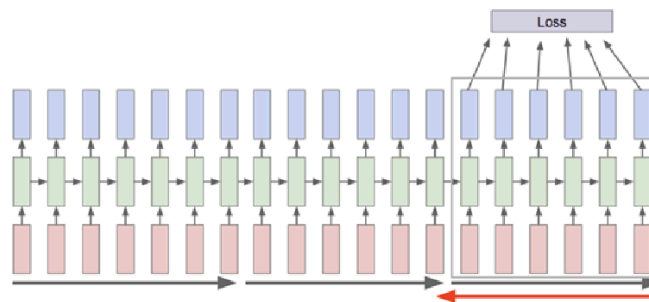
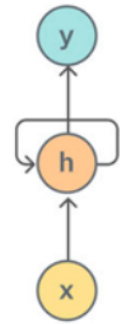
(-) **No pueden recordar estados previos.**

(-) **No manejan datos secuenciales** (datos donde el orden importa).



Redes Neuronales Recurrentes (RNN):

- 1) Adoptan **ciclos** en la estructura.
 - a) **Recuerdan la información** previa.
 - b) Procesan **datos secuenciales**.
 - i) El orden importa.
 - ii) Ejemplo → “No estoy feliz” ≠ “Estoy feliz”.
- 2) El estado actual de la red **depende del estado anterior y del estado presente**.
 - Un **estado** h_t es la memoria de la red en el momento t .
 - Un **paso** es el momento de la secuencia ($t = 1, 2, 3, \dots$).
- 3) Usan **siempre los mismos pesos** en cada paso → generan estados en cadena. Esto quiere decir que el estado h_t depende del estado anterior, h_{t-1} .
- 4) **Entrenamiento:**
 - a) Uso de **Backpropagation Through Time (BPTT)** → los gradientes se propagan para atrás a través de todas las etapas de la secuencia.
 - i) Es como un viaje al pasado.
 - ii) La red hace BPTT retrocediendo en los estados que se guardaron en el forward propagation.
 - b) Si la secuencia es muy larga, se usa **BPTT truncado** para que no sea tan costoso.

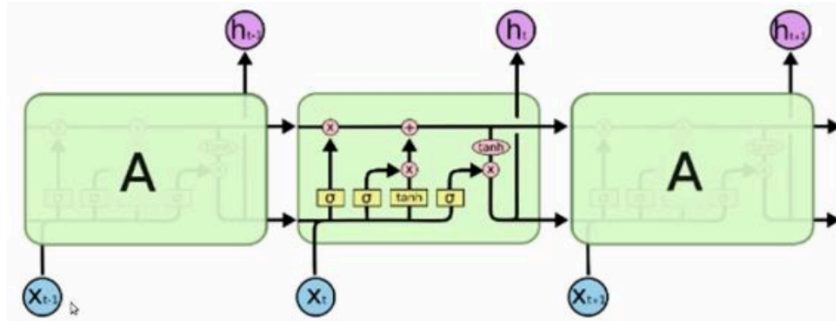


- 5) **Ventajas:**
 - a) Manejan secuencias de longitud variable.
 - Cuando el input no tiene siempre la misma cantidad de pasos: una oración puede tener más o menos palabras, un audio más o menos duración...
 - b) Tienen memoria interna.
 - c) Comparten pesos.
 - d) Pueden combinarse con CNNs.
- 6) **Desventajas:**
 - a) Entrenamiento **lento**.
 - b) Problemas → **vanishing gradient y exploding gradient**.
 - c) No capturan las **dependencias largas**.
 - Situaciones donde lo que pasa mucho antes afecta lo que pasa ahora.
 - Lo que pasa mucho antes suele olvidarse a lo largo de los pasos.
 - d) No escalan a mucha profundidad.

Long Short-Term Memory (LSTM):

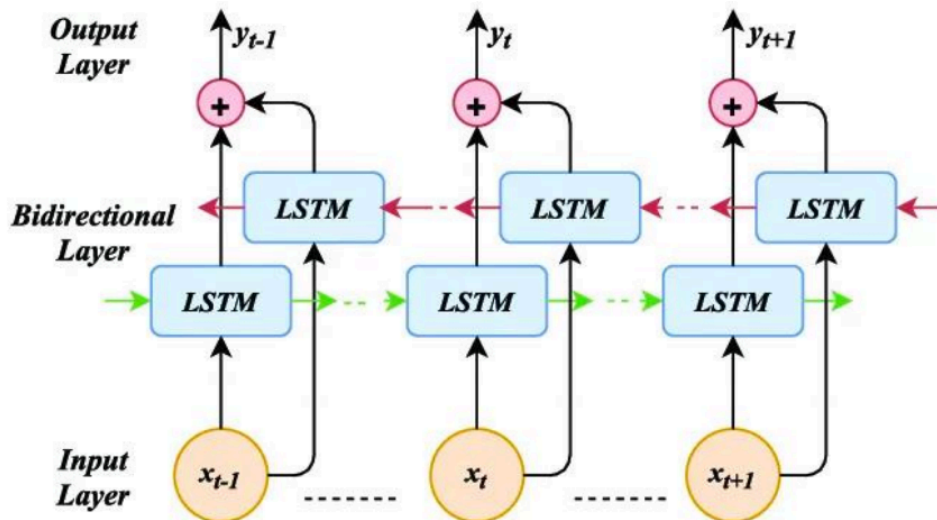
- 1) Es un tipo de RNN.
- 2) Diseñado para resolver el vanishing gradient y capturar dependencias largas.
- 3) Funcionamiento de tres puertas:
 - a) **Puerta de entrada** → decide **qué información nueva entra** a la memoria.

- b) **Puerta de olvido** → decide **qué información se descarta**.
- c) **Puerta de salida** → regula **qué parte de la memoria se usa como salida**.
Controla cuándo la memoria impacta en la red.
- 4) Uso del producto de Hadamard ($\odot$) → conserva información por largos períodos sin que se degrade.



Bidirectional LSTM (BiLSTM):

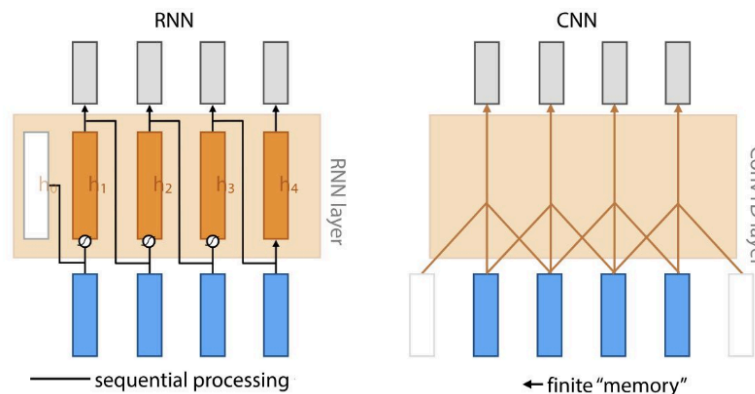
- 1) Procesa la información en **dos direcciones**:
 - a) **Forward** (pasado → futuro)
 - b) **Backward** (futuro → pasado)
- 2) Permite la incorporación del contexto completo → cuando una palabra tiene un significado según lo que venga después.



Clase XXXI: Transformadores

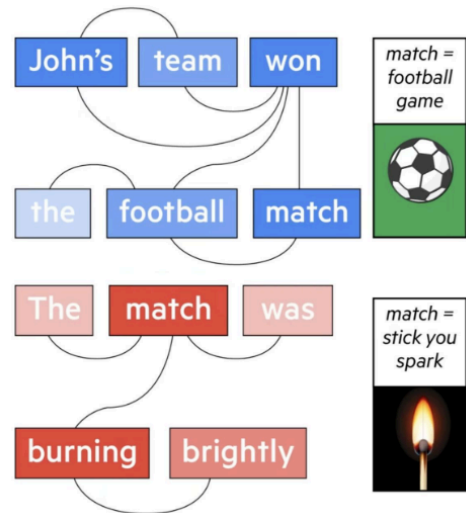
Las RNN podían procesar dependencias largas (lo siguen haciendo) pero tenían estas limitaciones:

- 1) Al operar secuencialmente, era un **proceso lento**.
- 2) La **información del pasado se “olvida”** con la longitud de la secuencia.



Mecanismo Attention:

- 1) Permite:
 - a) **Enfoque en lo importante** de la entrada.
 - b) Asignar **pesos de importancia** a cada token.
 - i) Un token es la unidad mínima en la que el modelo divide el texto.
 - ii) Puede ser una palabra ("casa"), un sub-token ("cas", "##a"), un signo de puntuación o un símbolo especial ([CLS], [MASK]).
 - c) **Procesamiento del contexto global**, sea corta o larga la secuencia.
- 2) Mira toda la secuencia a la vez → deja de ser secuencial.
- 3) Permite paralelismos → aumenta la eficiencia.
- 4) **Mecanismo** → un query **Q** se compara con muchos keys **K** para decidir cuáles valores **V** usar y con qué peso.
 - a) Este mecanismo puede ocurrir entre **dos secuencias distintas** o **dentro de la misma secuencia**.
 - b) "Lo que estoy procesando ahora, ¿con qué otra información debería relacionarlo?"



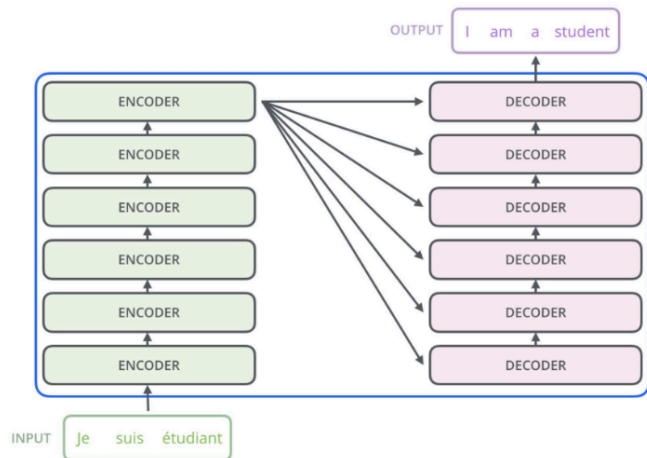
Mecanismo Self-Attention:

- 1) Usa el mismo mecanismo de Attention, solo con la restricción de que la secuencia se atiende a sí misma → cada token mira a todos los otros tokens de la misma oración.
- 2) Toma una palabra $\equiv$ token.
- 3) **Calcula:**
 - a) Query (**Q**) → "lo que estoy buscando".
 - i) Vector que representa **la pregunta** que hace un token al resto.
 - ii) Codifica **qué tipo de información necesita ese token** para entenderse.
 - iii) Si el token es "estaba" en la frase "El perro corrió al gato porque estaba cansado", el Query de "estaba" está preguntando: "¿A cuál palabra debería referirme? ¿'Perro' o 'gato'?"
 - b) Key (**K**) → "lo que cada token ofrece".
 - i) Cada token también tiene un Key, un vector que representa **la información disponible** que ese token puede aportar. Dice qué ofrece cada token a las consultas que se le hagan.
 - ii) La palabra "perro" tiene un Key que dice "soy un sustantivo, singular... por ahí puedo ser referente de 'estaba'".
 - iii) Los Keys avisan qué es lo que tienen, por ahí a algún otro token le sirve. Por eso, el Query de un token se compara con los Keys de todos los tokens. Si $Q \cdot K$ es grande, entonces el token es relevante.
 - c) Value (**V**) → "la información que se combina".

- i) Una vez que el Query decide a quién “prestar atención” usando los Keys, cada token aporta su **Value**.
- 4) **Resultado** → vector contextualizado para cada token.

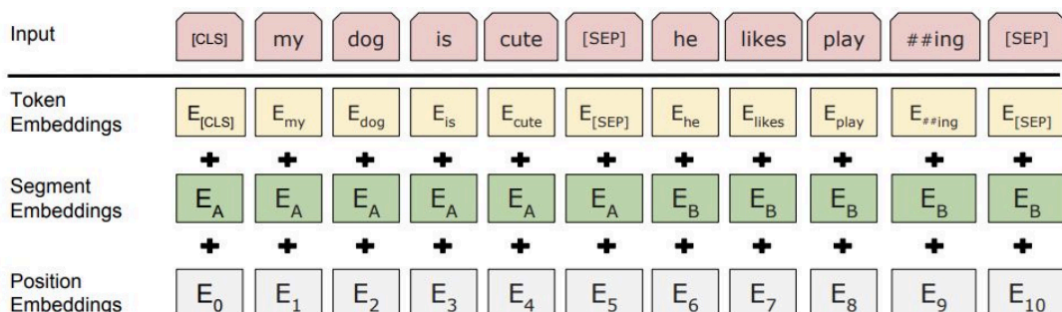
Transformers:

- 1) Reemplazo de las RNN.
- 2) **Arquitectura:**
 - a) 6 encoders apilados.
 - b) 6 decoders apilados.
- 3) Cada **encoder** tiene:
 - a) Self-Attention.
 - b) Red Neuronal Feedforward.
 - c) Conexiones residuales + LayerNorm.
- 4) Cada **decoder** tiene:
 - a) Masked Self-Attention.
 - b) Encoder-Decoder Attention.
 - c) Red Neuronal Feedforward.
 - d) Conexiones residuales.
- 5) Tiene **embeddings posicionales**:
 - a) El transformer no es un modelo secuencial.
 - b) Por lo tanto, se suman los embeddings posicionales para **indicar el orden de las palabras**.
- 6) **Paralelización** → cada palabra se puede procesar en paralelo dentro de cada capa, a diferencia de LSTM.
- 7) **Salida:**
 - a) El **decoder produce un vector** que luego pasa por:
 - i) Capa Lineal Fully-Connected.
 - ii) Softmax → generar una probabilidad por palabra del vocabulario.
 - b) El modelo **selecciona la palabra con mayor probabilidad** para cada paso de decodificación.



Bidirectional Encoder Representations from Transformers (BERT):

- 1) Modelo **basado solo en encoders**.
- 2) **Bidireccionalidad** → análisis de contexto a **izquierda y derecha en simultáneo**.
- 3) **Pre-entrenamiento:**
 - a) Primera tarea → **Masked Language Modeling (MLM)**
 - i) Se tapan palabras con [MASK].
 - ii) El modelo debe predecir las palabras tapadas.
 - b) Segunda tarea → **Next Sentence Prediction (NSP)**
 - i) Determina si una oración sigue a otra.
- 4) **Fine-Tuning:**
 - a) Con el preentrenamiento listo.
 - b) Se ajusta el modelo a tareas específicas → clasificación, QA, NER...



Vision Transformer (ViT)

- 1) La imagen se divide en patches.
- 2) Cada patch se convierte en un embedding.
- 3) Se le aplica Transformer como si fuera una secuencia de tokens visuales.

Clase XXXII: Ética

Bias → cuando un modelo produce resultados sistemáticamente injustos o desbalanceados debido a cómo fue entrenado o diseñado. Aparece en humanos y también en IA.

Ejemplos de biases en IA:

- 1) **Apple Card** discriminando por género en asignación de crédito.
- 2) **Facebook** diciendo que los negros son monos.
- 3) **Amazon** penalizando mujeres en un sistema de reclutamiento.
- 4) **Admisiones universitarias** penalizando personas no europeas.

Sesgo en cada etapa de la IA:

- 1) **Datos** → desbalance, selección incorrecta, datos no representativos.
- 2) **Modelo** → mala interpretabilidad y métricas que no capturan injusticias.
- 3) **Entrenamiento y deployment** → feedback loops que refuerzan desigualdades.
- 4) **Evaluación** → análisis así no más en subgrupos.
→ El modelo anda bien en general pero mal en las minorías.
- 5) **Interpretación humana** → sesgos de quienes toman las decisiones.

No alcanza con que nuestro modelo tenga 99% de precisión, se tiene que evaluar la **justicia**, la **interpretabilidad**, la **robustez a manipulación** y la **responsabilidad**.

¿De dónde vienen los sesgos?

- 1) **Data-driven:**
 - a) **Selection bias** → el dataset **no representa a la población real**.
→ Pasa **cuando seleccionamos los datos**.
→ “En un dataset de detección facial hay más blancos que negros”.
→ El modelo funcionará bien en los blancos.
 - b) **Reporting bias** → el dataset **no refleja bien la realidad**.
→ Pasa **cuando se describen o se reportan los datos**.
→ “En internet se habla más de Boca que de River, pero en realidad nadie habla de Boca.”
→ El modelo aprende una realidad paralela.
 - c) **Sampling bias** → algunos datos aparecen más que otros.
→ Se desbalancea el dataset porque algunas muestras son más frecuentes.
→ “En un dataset urbano hay más fotos de autos y edificios que en una zona rural.
- 2) **Interpretation-given:**
 - a) **Correlation Fallacy** → el modelo entiende que dos variables correlacionadas implican causalidad, esto es falso.
→ El modelo **toma decisiones erradas por tener correlaciones accidentales**.

- “El modelo entiende que vivir en el centro causa vivir en un edificio”, pero en realidad es un factor ajeno al centro.
- b) **Overgeneralization** → el modelo saca conclusiones generales a partir de datos limitados.
 - El modelo **funciona bien en su dataset y generaliza con eso**. Falla en contextos que no estudió.
 - “El modelo de reconocimiento facial va a andar bien para todos” cuando lo entrenó solo con asiáticos.
- c) **Automation Bias** → los humanos confiamos mucho en decisiones de sistemas automáticos.
 - El sesgo no solo está en los datos, sino en nuestra interpretación.
 - “Los médicos aceptan un diagnóstico que les dio una IA porque creen que es un ser superior a nosotros”.
 - Los humanos asumimos que las máquinas no se equivocan.

Métodos de mitigación → **predicción conjunta de label y variable sensible, negación de gradiente** para que el modelo use la variable sensible, **rebalanceo del dataset** y **fairness constraints**.